

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
```



Search



MergeSort1.cpp X

```
void merge(int arr[], int left, int mid, int right) {  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
    }  
}  
  
int main() {  
    int N;  
    cin >> N;  
    int arr[1000];  
    for(int i=0; i<N; i++){  
        cin >> arr[i];  
    }  
  
    mergeSort(arr, 0, N-1);  
    for(int i=0; i<N; i++){  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Q Search

ASUS Vivobook


```
void merge(int arr[], int left, int mid, int right) {  
    // Merge the two sorted sub-arrays into a temporary array  
    // and then copy it back to the original array  
}
```

```
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
    }  
}
```

```
int main() {  
    int N;  
    cin >> N;  
    int arr[1000];  
    for (int i = 0; i < N; i++) {  
        cin >> arr[i];  
    }  
}
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ MergeSort1.cpp

PS C:\PD\DSA\Searching> ./a.exe

6

450 1200 800 650 300 1500

300 450 650 800 1200 1500

PS C:\PD\DSA\Searching> █



Search



ASUS V


```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
```



Search




```
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
  
        merge(arr, left, mid, right);  
    }  
}
```

```
int main() {  
    int N;  
    cin >> N;  
    int arr[1000];  
    for (int i = 0; i < N; i++) {  
        cin >> arr[i];  
    }  
  
    mergeSort(arr, 0, N - 1);  
    for (int i = 0; i < N; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    double median;  
    if (N % 2 == 0)  
        median = (arr[N/2 - 1] + arr[N/2]) / 2.0;  
    else  
        median = arr[N/2];  
    cout << "Median:" << median << endl;  
  
    int count = 0;  
    for (int i = 0; i < N; i++) {  
        if (arr[i] > median)  
            count++;  
    }  
}
```



```
64     if (N % 2 == 0)
        median = (arr[N/2 - 1] + arr[N/2]) / 2.0;
    else
        median = arr[N/2];
    cout << "Median:" << median << endl;

    int count = 0;
    for (int i = 0; i < N; i++) {
        if (arr[i] > median)
            count++;
    }
    cout << "Orders above Median:" << count << endl;

    int difference = arr[N - 1] - arr[0];
    cout << "Difference:" << difference;

    return 0;
}
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ MergeSort2.cpp

PS C:\PD\DSA\Searching> ./a.exe

7

12 25 18 30 15 22 40

12 15 18 22 25 30 40

Median:22

Orders above Median:3

Difference:28

PS C:\PD\DSA\Searching> █



Search



ASUS Vivobook

quickSort1.cpp X

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
}
```

14

```
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

15

16

17

18

19

20

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

21

22

23

24

25

26

27

28

29

```
int main() {
    int n;
    cin >> n;
    int arr[1000];

    for (int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);
}
```

30

31

32

33

34

35

36

37



Q Search



ASUS Vivobook

F4



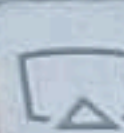
F5



F6



F7



F8



F9


```
void quickSort(int arr[], int low, int high) {  
  
int main() {  
    int n;  
    cin >> n;  
    int arr[1000];  
  
    for (int i = 0; i < n; i++)  
        cin >> arr[i];  
  
    quickSort(arr, 0, n - 1);  
  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
  
    return 0;  
}
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ quickSort1.cpp

PS C:\PD\DSA\Searching> ./a.exe

6

120 45 78 200 60 95

45 60 78 95 120 200

PS C:\PD\DSA\Searching> █



Search




```
#include <iostream>
using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

10 int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}

25 void quickSort(int arr[], int low, int high) {
26     if (low < high) {
27         int pi = partition(arr, low, high);
28
29         quickSort(arr, low, pi - 1);
30         quickSort(arr, pi + 1, high);
31     }
32 }

34 int main() {
35     int N;
36     cin >> N;
37 }
```



Search



ASUS Vivobook


```
int arr[100000];
int sum = 0;

for (int i = 0; i < N; i++) {
    cin >> arr[i];
    sum += arr[i];
}

quickSort(arr, 0, N - 1);

for (int i = 0; i < N; i++)
    cout << arr[i] << " ";
cout << endl;

cout << "Top 5: ";
int topSum = 0;
for (int i = 0; i < 5; i++) {
    cout << arr[i] << " ";
    topSum += arr[i];
}
cout << endl;

cout << "Average of Top 5: " << topSum / 5 << endl;

double overallAvg = (double)sum / N;

int count = 0;
for (int i = 0; i < N; i++) {
    if (arr[i] > overallAvg)
        count++;
}

cout << "Values Above Overall Average: " << count;

return 0;
}
```



Search



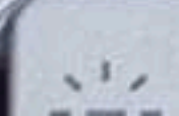
ASUS Vivobook



F5



F6



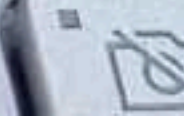
F7



F8



F9




```
int sum = 0;
```

```
for (int i = 0; i < N; i++) {  
    cin >> arr[i];  
    sum += arr[i];  
}
```

```
quickSort(arr, 0, N - 1);
```

```
for (int i = 0; i < N; i++)  
    cout << arr[i] << " ";  
cout << endl;
```

```
cout << "Top 5: ";
```

```
int topSum = 0;
```

```
for (int i = 0; i < 5; i++) {  
    cout << arr[i] << " ";  
    topSum += arr[i];  
}
```

```
cout << endl;
```

```
cout << "Average of Top 5: " << topSum / 5
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ quickSort2.cpp

PS C:\PD\DSA\Searching> ./a.exe

8

500 1200 700 2000 900 1500 3000 2500

3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Values Above Overall Average: 3

PS C:\PD\DSA\Searching> █



Search



ASUS Viv


```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

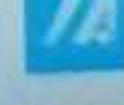
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    cin >> n;
    int arr[1000];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
}
```



Search



ASUS Vivobook


```
int n;  
cin >> n;  
int arr[1000];  
for (int i = 0; i < n; i++)  
    cin >> arr[i];  
  
heapSort(arr, n);  
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";  
  
return 0;  
44 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\PD\DSA\Searching> g++ heapSort1.cpp

PS C:\PD\DSA\Searching> ./a.exe

6

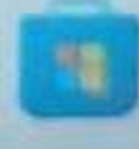
850 1200 950 600 1500 1000

600 850 950 1000 1200 1500

PS C:\PD\DSA\Searching> █



Search



ASUS Vivobook

F4

F5

F6

F7

F8


```
#include <iostream>
#include <iomanip>
using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        heapify(arr, n, smallest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

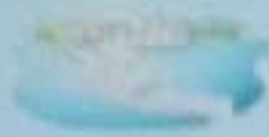


```
void heapSort(int arr[], int n) {  
    for (int i = n - 1; i > 0; i--) {  
        heapify(arr, i, n);  
    }  
  
    for (int i = 0; i < n / 2; i++)  
        swap(arr[i], arr[n - i - 1]);  
}
```

```
int main() {  
    int N;  
    cin >> N;  
  
    int arr[100000];  
    double sum = 0;  
  
    for (int i = 0; i < N; i++) {  
        cin >> arr[i];  
        sum += arr[i];  
    }  
  
    heapSort(arr, N);  
  
    for (int i = 0; i < N; i++)  
        cout << arr[i] << " ";  
    cout << endl;  
  
    cout << "Fastest: " << arr[0] << endl;  
    cout << "Slowest: " << arr[N - 1] << endl;  
  
    double avg = sum / N;  
    cout << "Average: " << avg << endl;  
  
    int count = 0;  
    for (int i = 0; i < N; i++) {  
        if (arr[i] < avg)  
            count++;  
    }  
}
```



Search




```
cout << endl;

cout << "Fastest: " << arr[0] << endl;
cout << "Slowest: " << arr[N - 1] << endl;

double avg = sum / N;
cout << "Average: " << avg << endl;

int count = 0;
for (int i = 0; i < N; i++) {
    if (arr[i] < avg)
        count++;
}

cout << "Cases Faster Than Average: " << count << endl;

double percent = (count * 100.0) / N;
cout << fixed << setprecision(2);
cout << "Percentage: " << percent << "%";

return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\PD\DSA\Searching> g++ heapSort2.cpp

PS C:\PD\DSA\Searching> ./a.exe

8

12 7 15 9 20 5 10 8

5 7 8 9 10 12 15 20

Fastest: 5

Slowest: 20

Average: 10.75

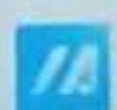
Cases Faster Than Average: 5

Percentage: 62.50%

PS C:\PD\DSA\Searching> |

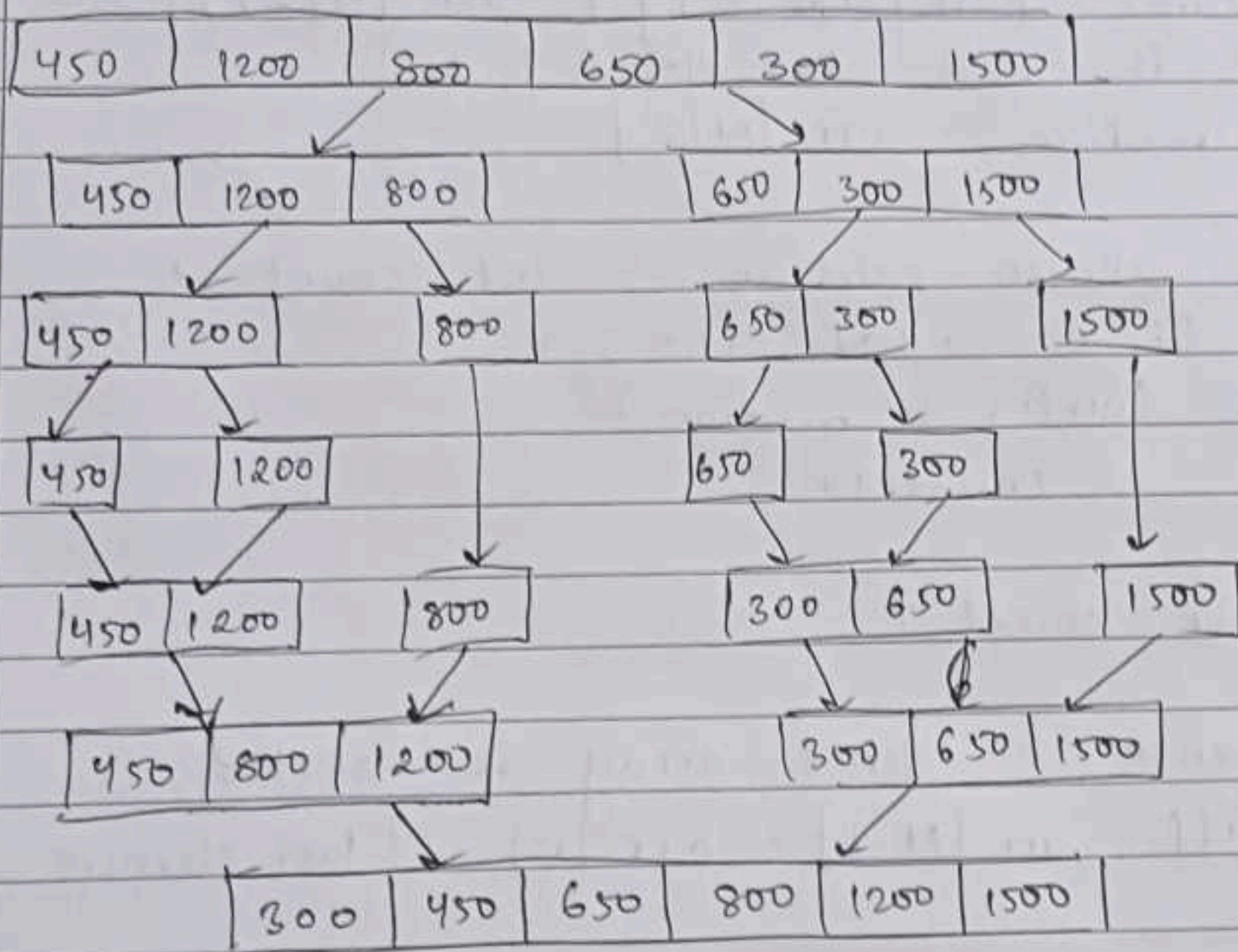


Search

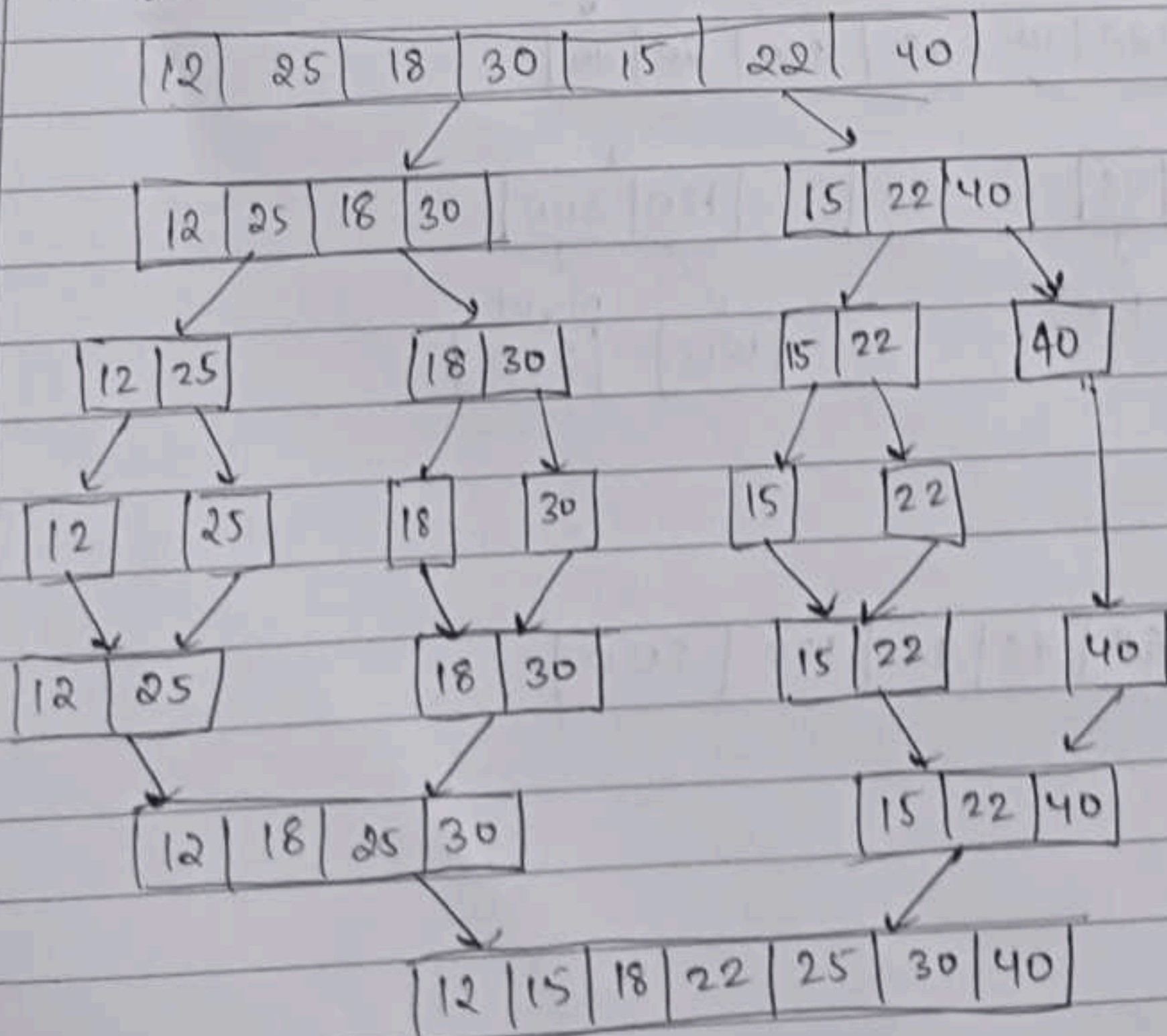


ASUS Vivobook

1) Merge Sort : Easy



2) Merge Sort : Medium



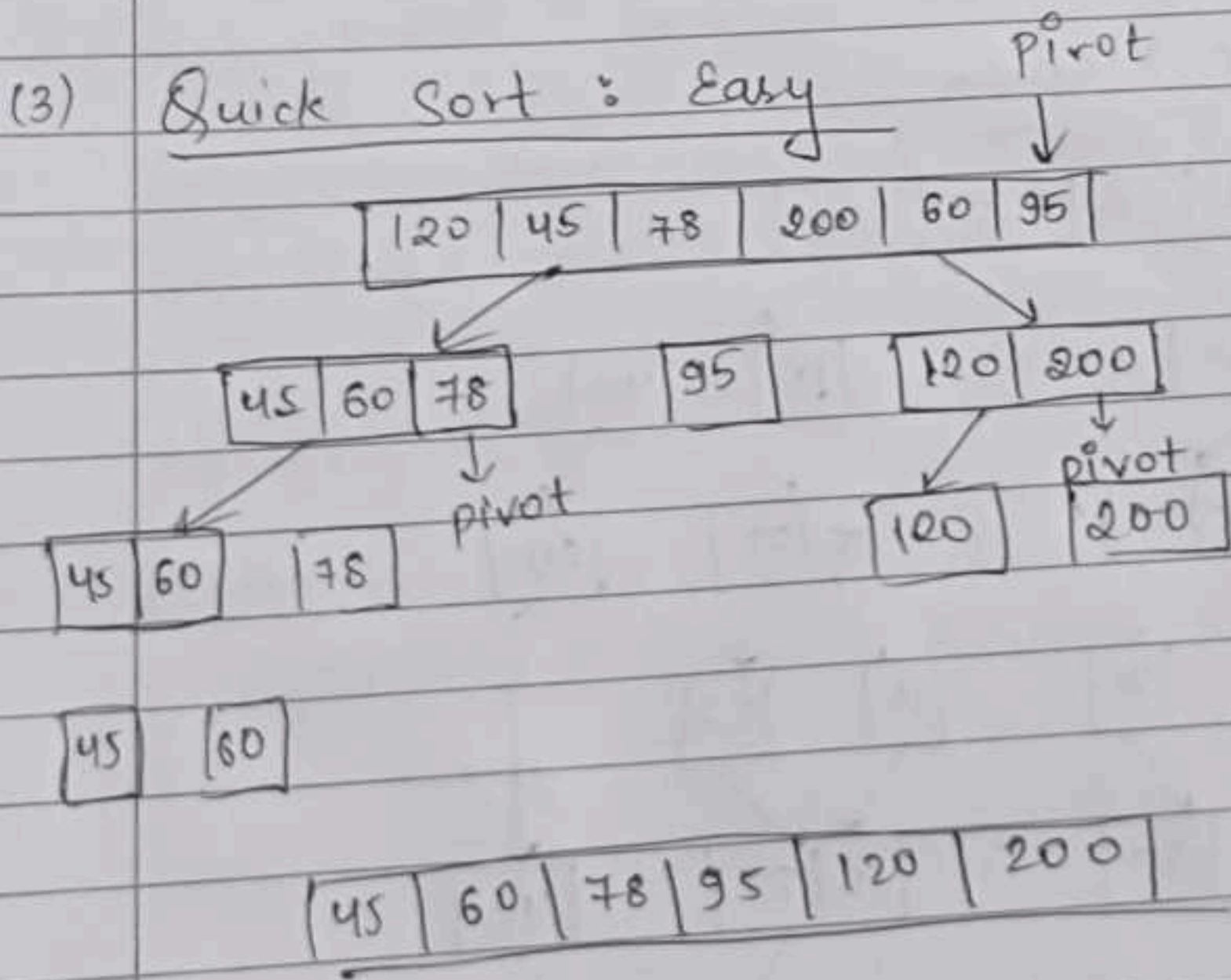
- median :-
(1) if N is even ($N \% 2 == 0$)
$$\text{median} = (\text{arr}[N/2 - 1] + \text{arr}[N/2]) / 2.0$$

(2) if N is odd ($N \% 2 != 0$)
$$\text{median} = \text{arr}[N/2]$$

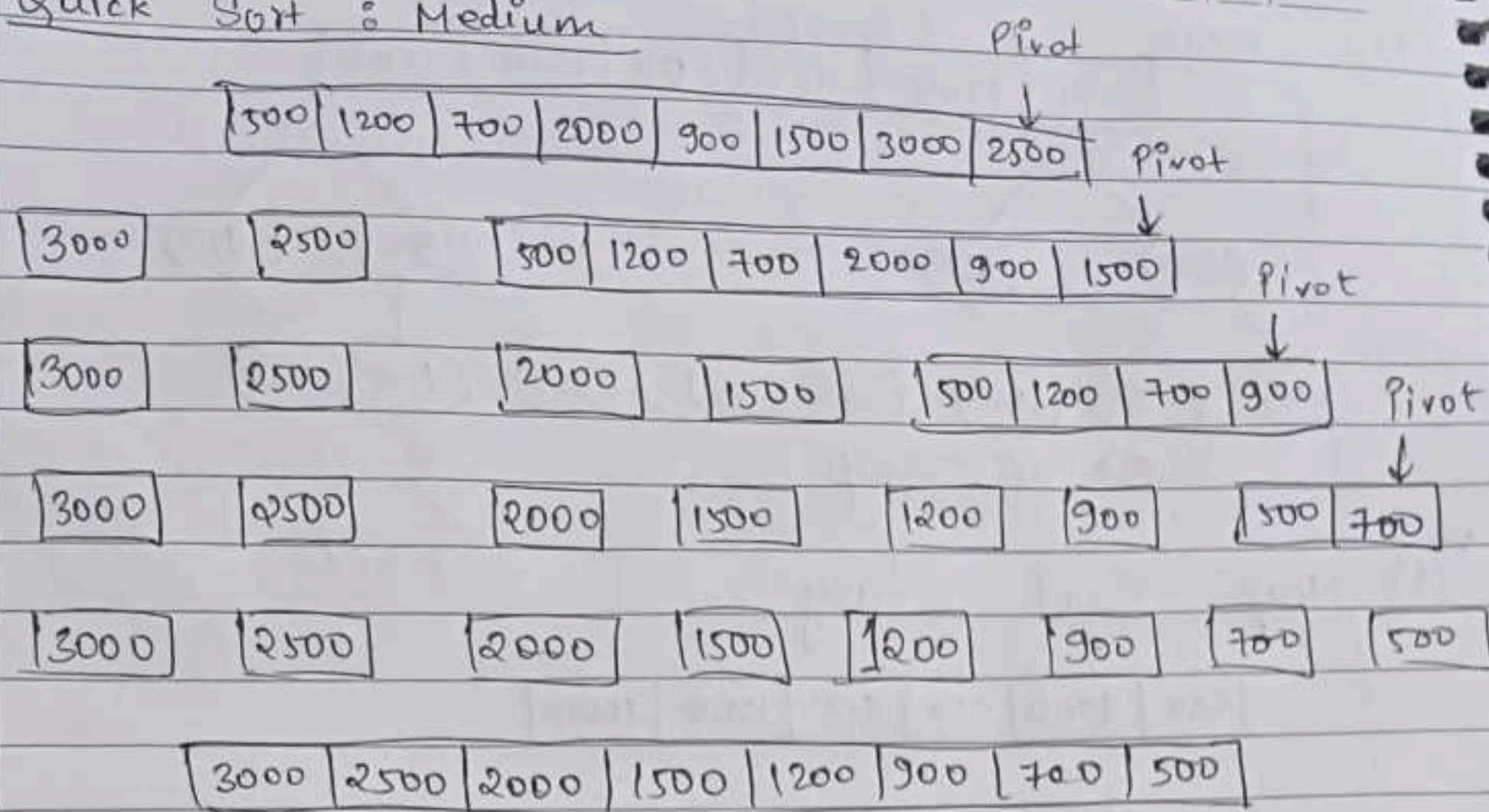
- orders above median :-

```
int count = 0;  
for (i = 0; i < N; i++) {  
    if (arr[i] > median)  
        count++;  
}  
return count;
```

- Difference :- Since array is sorted,
$$\text{diff} = \text{arr}[N-1] - \text{arr}[0]$$
 (last element - first element)



(4) Quick Sort : Medium



- Top 5 :-

```
for (i = 0; i < 5; i++) {  
    cout << arr[i];  
}
```
- Top 5 sum :-

```
for (i = 0; i < 5; i++) {  
    topSum += arr[i];  
}
```
- Top 5 average :- $avg = topSum / 5;$
- Overall avg :-

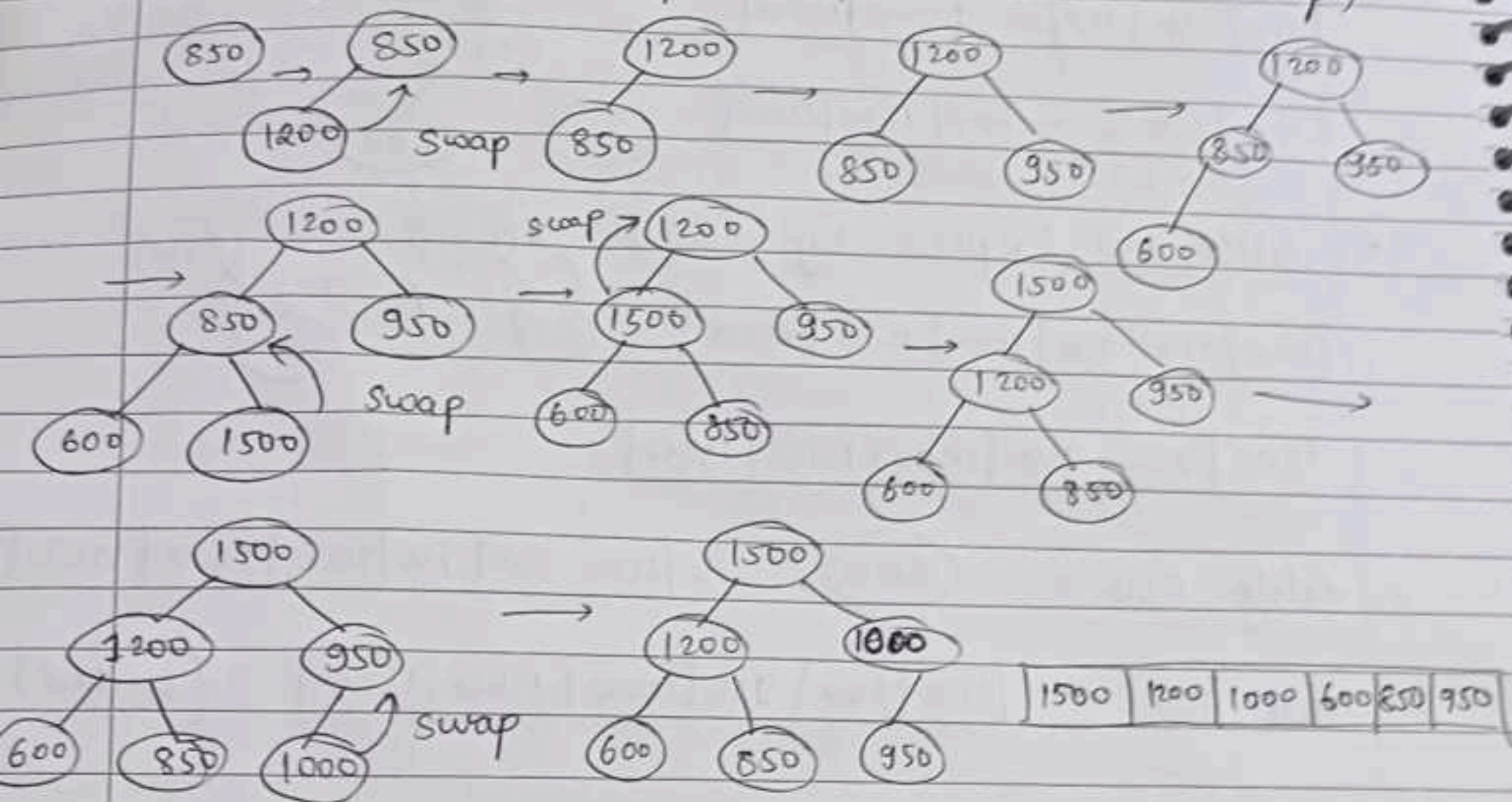
```
for (i = 0; i < n; i++) {  
    sum += arr[i];  
}  
totalavg = sum / n;
```
- Values above overall average :-

```
int count = 0;  
for (int i = 0; i < n; i++) {  
    if (arr[i] > totalavg)  
        count++;  
}  
return count;
```


(5) Heap Sort : Easy

850 | 1200 | 950 | 600 | 1500 | 1000

• heapify :- max. heap (if parent < child $\Rightarrow$ swap)

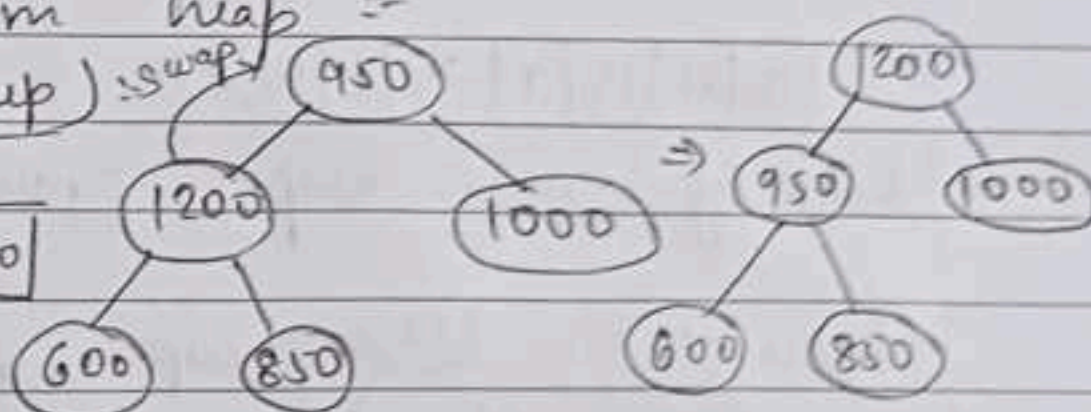


$\Rightarrow$ deleting elements from heap :-

• 1500 delete (-950 takes up) : swap

950 | 1200 | 1000 | 600 | 850 | 1500

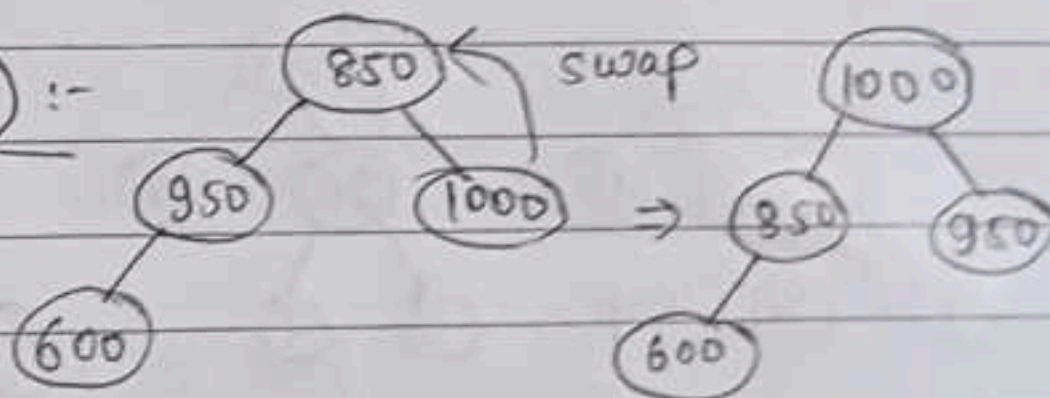
1200 | 950 | 1000 | 600 | 850 | 1500



• delete 1200 (replace by 850) :-

850 | 950 | 1000 | 600 | 1200 | 1500

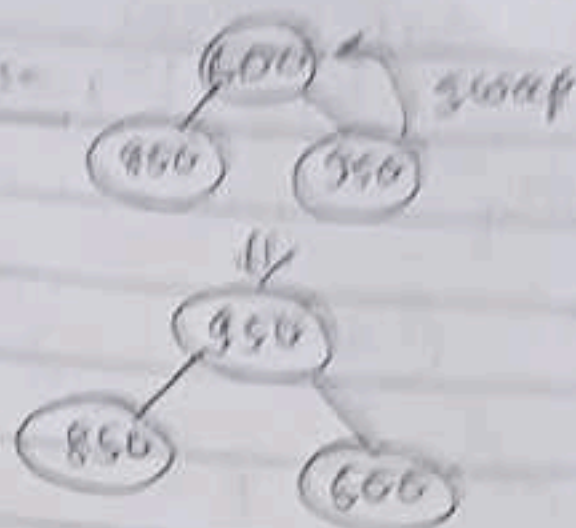
1000 | 850 | 950 | 600 | 1200 | 1500



- delete 1000 (replace by 600) =

600	850	950	1000	1200	1300
-----	-----	-----	------	------	------

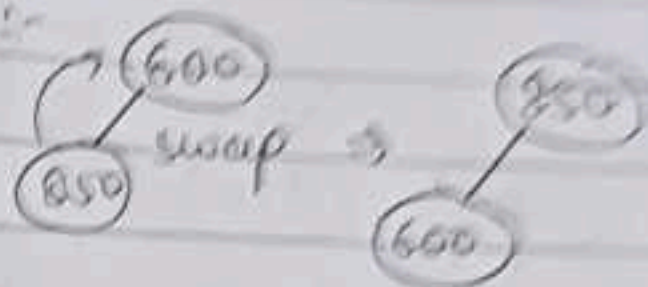
950	850	600	1000	1200	1500
-----	-----	-----	------	------	------



• delete 950 (replace by 600) :-

600	850	950	1000	1200	1500
-----	-----	-----	------	------	------

850	600	950	1000	1200	1500
-----	-----	-----	------	------	------



1	delete	850 :-
---	--------	--------

600

600	850	950	1000	1200	1500
-----	-----	-----	------	------	------

1	Delete 600	5
---	------------	---

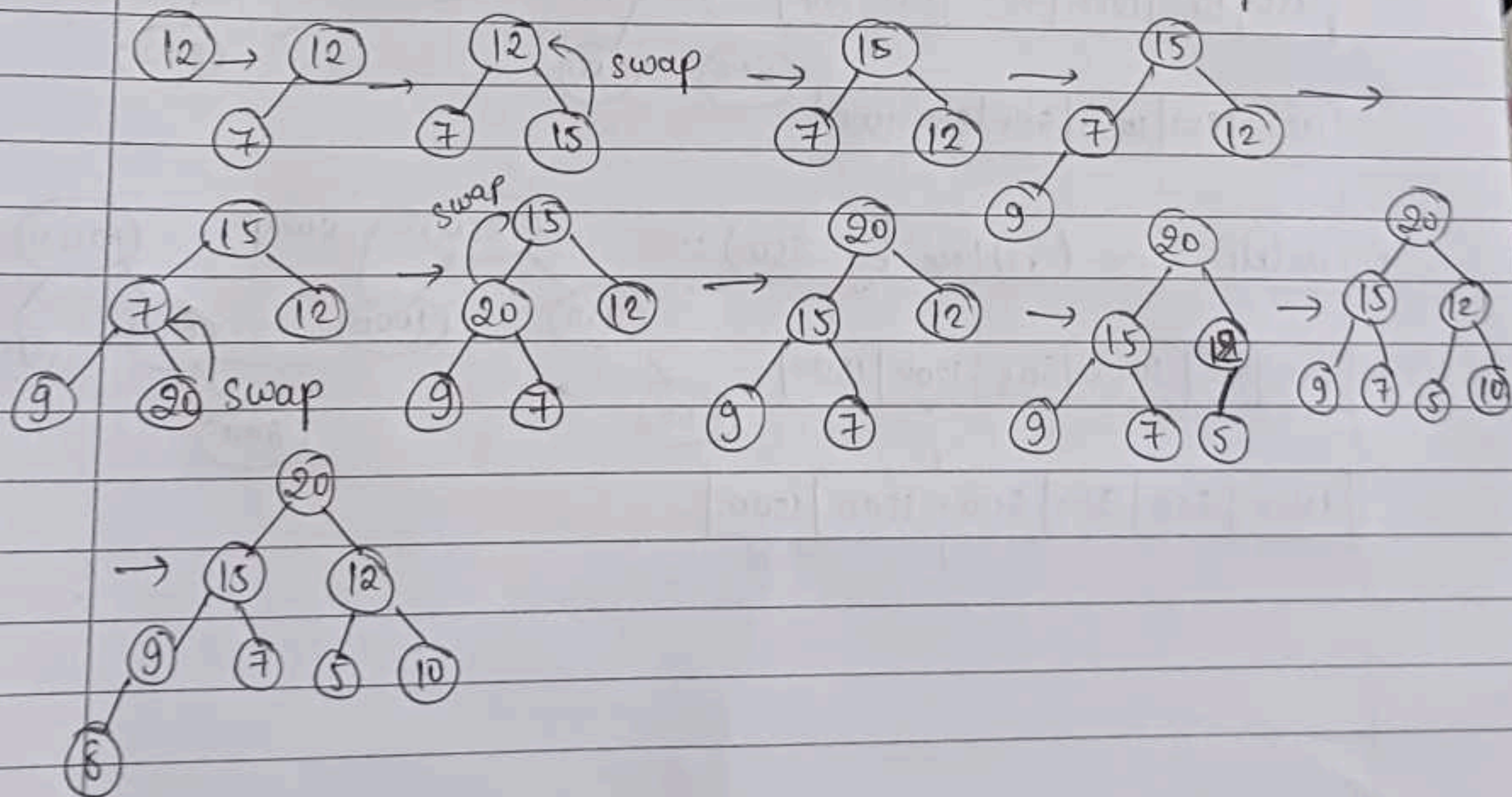
600	850	950	1000	1200	1500
-----	-----	-----	------	------	------

(sauté)

(6)	Heap Sort : Medium
-----	--------------------

12	7	15	9	20	5	10	8
----	---	----	---	----	---	----	---

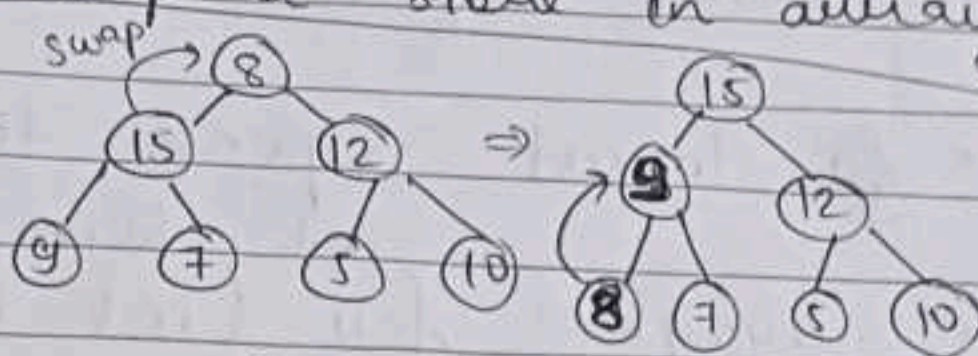
- heapify : max. heap $\Rightarrow$ (parent ~~>~~ child) $\Rightarrow$ swap



• delete elements from heap & store in array.

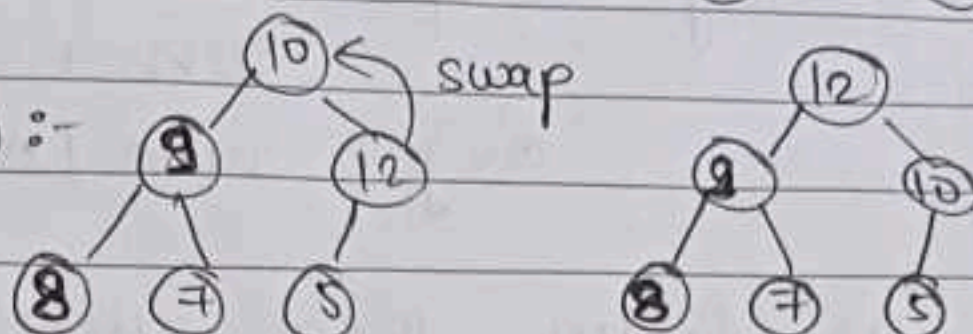
⇒ delete 20 (replace by 8) :-

15 | 9 | 12 | 8 | 7 | 5 | 10 | 20



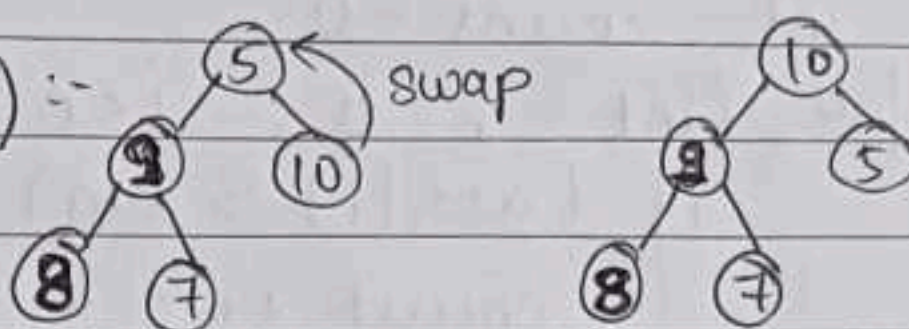
⇒ delete 15 (replace by 10) :-

12 | 9 | 10 | 8 | 7 | 5 | 15 | 20



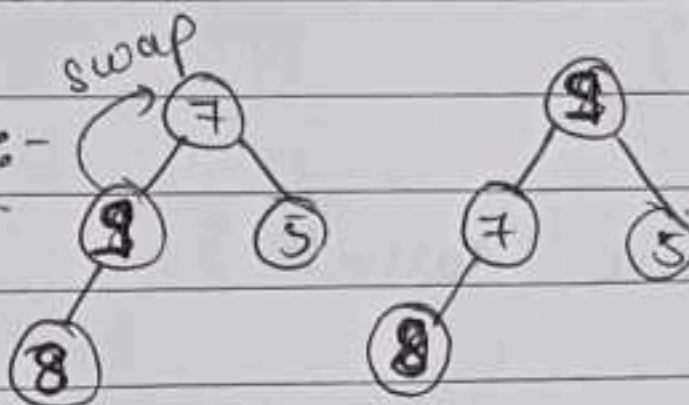
⇒ delete 12 (replace by 5) :-

10 | 9 | 5 | 8 | 7 | 12 | 15 | 20



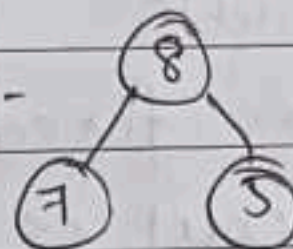
⇒ delete 10 (replace by 7) :-

9 | 7 | 5 | 8 | 10 | 12 | 15 | 20



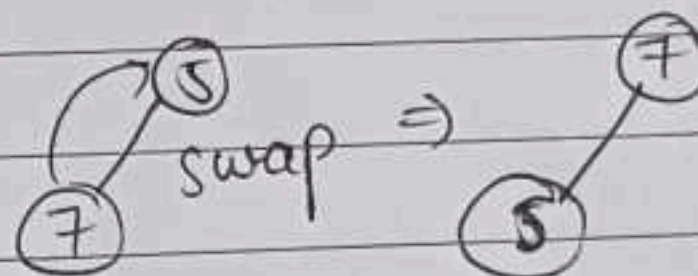
⇒ delete 9 (replace by 8) :-

8 | 7 | 5 | 9 | 10 | 12 | 15 | 20



⇒ delete 8 (replace by 5) :-

7 | 5 | 8 | 9 | 10 | 12 | 15 | 20



⇒ delete 7 :-

5 | 7 | 8 | 9 | 10 | 12 | 15 | 20

⇒ delete 5 :-

5 | 7 | 8 | 9 | 10 | 12 | 15 | 20

- fastest response time = $\text{arr}[0] = 5$

- slowest response time = $\text{arr}[n-1] = 80$

- average :-

```
int sum = 0;  
for (int i = 0; i < n; i++)  
    sum += arr[i]  
avg = sum / n;
```

- cases faster than average :-

```
int count = 0;  
for (int i = 0; i < n; i++)  
    if (arr[i] > avg)  
        count++;  
return count;
```

- Percentage :-

```
double percent = (count * 100.0) / n;  
cout << fixed << setprecision(2);  
cout << percent;
```